

Follow the Gradient

Differentiable Matrix Completion

and a million-dollar prediction problem

Jonathon Hare

Vision, Learning and Control
University of Southampton

Recap: Singular Value Decomposition and its applications

Let's now change direction — we're going to look at an early success story resulting from using some differentiation and the Singular Value Decomposition (SVD).

For complex $\mathbf{A}$:

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^*$$

where $\mathbf{V}^*$ is the *conjugate transpose* of $\mathbf{V}$.

For real $\mathbf{A}$:

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^{\top}$$

Recap: Singular Value Decomposition and its applications

- SVD has many uses:
 - Computing the Eigendecomposition:
 - Eigenvectors of $\mathbf{A}\mathbf{A}^\top$ are columns of $\mathbf{U}$,
 - Eigenvectors of $\mathbf{A}^\top\mathbf{A}$ are columns of $\mathbf{V}$,
 - and the non-zero values of $\mathbf{\Sigma}$ are the square roots of the non-zero eigenvalues of both $\mathbf{A}\mathbf{A}^\top$ and $\mathbf{A}^\top\mathbf{A}$.

Recap: Singular Value Decomposition and its applications

- SVD has many uses:
 - Computing the Eigendecomposition:
 - Eigenvectors of $\mathbf{A}\mathbf{A}^\top$ are columns of $\mathbf{U}$,
 - Eigenvectors of $\mathbf{A}^\top\mathbf{A}$ are columns of $\mathbf{V}$,
 - and the non-zero values of $\mathbf{\Sigma}$ are the square roots of the non-zero eigenvalues of both $\mathbf{A}\mathbf{A}^\top$ and $\mathbf{A}^\top\mathbf{A}$.
 - Dimensionality reduction
 - ...use to compute PCA

Recap: Singular Value Decomposition and its applications

- SVD has many uses:
 - Computing the Eigendecomposition:
 - Eigenvectors of $\mathbf{A}\mathbf{A}^\top$ are columns of $\mathbf{U}$,
 - Eigenvectors of $\mathbf{A}^\top\mathbf{A}$ are columns of $\mathbf{V}$,
 - and the non-zero values of $\mathbf{\Sigma}$ are the square roots of the non-zero eigenvalues of both $\mathbf{A}\mathbf{A}^\top$ and $\mathbf{A}^\top\mathbf{A}$.
 - Dimensionality reduction
 - ...use to compute PCA
 - Computing the Moore-Penrose Pseudoinverse
 - for real $\mathbf{A}$: $\mathbf{A}^+ = \mathbf{V}\mathbf{\Sigma}^+\mathbf{U}^\top$ where $\mathbf{\Sigma}^+$ is formed by taking the reciprocal of every non-zero diagonal element and transposing the result.

Recap: Singular Value Decomposition and its applications

- SVD has many uses:
 - Computing the Eigendecomposition:
 - Eigenvectors of $\mathbf{A}\mathbf{A}^\top$ are columns of $\mathbf{U}$,
 - Eigenvectors of $\mathbf{A}^\top\mathbf{A}$ are columns of $\mathbf{V}$,
 - and the non-zero values of $\mathbf{\Sigma}$ are the square roots of the non-zero eigenvalues of both $\mathbf{A}\mathbf{A}^\top$ and $\mathbf{A}^\top\mathbf{A}$.
 - Dimensionality reduction
 - ...use to compute PCA
 - Computing the Moore-Penrose Pseudoinverse
 - for real $\mathbf{A}$: $\mathbf{A}^+ = \mathbf{V}\mathbf{\Sigma}^+\mathbf{U}^\top$ where $\mathbf{\Sigma}^+$ is formed by taking the reciprocal of every non-zero diagonal element and transposing the result.

Recap: Singular Value Decomposition and its applications

- SVD has many uses:
 - Computing the Eigendecomposition:
 - Eigenvectors of $\mathbf{A}\mathbf{A}^\top$ are columns of $\mathbf{U}$,
 - Eigenvectors of $\mathbf{A}^\top\mathbf{A}$ are columns of $\mathbf{V}$,
 - and the non-zero values of Σ are the square roots of the non-zero eigenvalues of both $\mathbf{A}\mathbf{A}^\top$ and $\mathbf{A}^\top\mathbf{A}$.
 - Dimensionality reduction
 - ...use to compute PCA
 - Computing the Moore-Penrose Pseudoinverse
 - for real $\mathbf{A}$: $\mathbf{A}^+ = \mathbf{V}\Sigma^+\mathbf{U}^\top$ where Σ^+ is formed by taking the reciprocal of every non-zero diagonal element and transposing the result.
 - Low-rank approximation
 - if you take the ρ columns of $\mathbf{U}$, and the ρ rows of $\mathbf{V}^\top$ corresponding to the ρ largest singular values, you can form the matrix $\mathbf{A}_\rho = \mathbf{U}_\rho \Sigma_\rho \mathbf{V}_\rho^\top$ which will be the *best* rank- ρ approximation of the original $\mathbf{A}$ in terms of the Frobenius norm.

The Netflix Challenge

A million-dollar prediction problem

- In 2006, Netflix released a large collection of customer ratings and offered \$1 million for improving its recommender system by 10%.

The Netflix Challenge

A million-dollar prediction problem

- In 2006, Netflix released a large collection of customer ratings and offered \$1 million for improving its recommender system by 10%.
- The released data contained:

The Netflix Challenge

A million-dollar prediction problem

- In 2006, Netflix released a large collection of customer ratings and offered \$1 million for improving its recommender system by 10%.
- The released data contained:
 - 100,480,507 ratings;

The Netflix Challenge

A million-dollar prediction problem

- In 2006, Netflix released a large collection of customer ratings and offered \$1 million for improving its recommender system by 10%.
- The released data contained:
 - 100,480,507 ratings;
 - from 480,189 customers;

The Netflix Challenge

A million-dollar prediction problem

- In 2006, Netflix released a large collection of customer ratings and offered \$1 million for improving its recommender system by 10%.
- The released data contained:
 - 100,480,507 ratings;
 - from 480,189 customers;
 - for 17,770 movies.

The Netflix Challenge

A million-dollar prediction problem

- In 2006, Netflix released a large collection of customer ratings and offered \$1 million for improving its recommender system by 10%.
- The released data contained:
 - 100,480,507 ratings;
 - from 480,189 customers;
 - for 17,770 movies.
- Given a customer, a movie and everything that customer had rated previously: **what rating would they give the movie?**

The Netflix Challenge

The user–movie rating matrix

	M_1	M_2	M_3	M_4	M_5	M_6
U_1	5	.	2	.	4	.
U_2	.	3	.	5	.	4
U_3	4	.	1	.	.	?
U_4	.	2	.	4	5	.
U_5	1	.	.	2	.	1

- Rows represent users.
- Columns represent movies.

The complete matrix would contain about 8.5 billion entries, but only about 1.2% of them were observed.

The Netflix Challenge

The user–movie rating matrix

	M_1	M_2	M_3	M_4	M_5	M_6
U_1	5	.	2	.	4	.
U_2	.	3	.	5	.	4
U_3	4	.	1	.	.	?
U_4	.	2	.	4	5	.
U_5	1	.	.	2	.	1

- Rows represent users.
- Columns represent movies.
- Entries are ratings from 1 to 5.

The complete matrix would contain about 8.5 billion entries, but only about 1.2% of them were observed.

The Netflix Challenge

The user–movie rating matrix

	M_1	M_2	M_3	M_4	M_5	M_6
U_1	5	.	2	.	4	.
U_2	.	3	.	5	.	4
U_3	4	.	1	.	.	?
U_4	.	2	.	4	5	.
U_5	1	.	.	2	.	1

- Rows represent users.
- Columns represent movies.
- Entries are ratings from 1 to 5.
- A dot is *unobserved*; it is not a rating of zero.

The complete matrix would contain about 8.5 billion entries, but only about 1.2% of them were observed.

The Netflix Challenge

Why might a low-rank model work?

- Movies are not completely unrelated objects.

The Netflix Challenge

Why might a low-rank model work?

- Movies are not completely unrelated objects.
 - They have properties such as genre, style, cast, age and overall popularity.

The Netflix Challenge

Why might a low-rank model work?

- Movies are not completely unrelated objects.
 - They have properties such as genre, style, cast, age and overall popularity.
- Users are not completely unrelated either.

The Netflix Challenge

Why might a low-rank model work?

- Movies are not completely unrelated objects.
 - They have properties such as genre, style, cast, age and overall popularity.
- Users are not completely unrelated either.
 - They have different preferences for those properties, as well as different tendencies in how they use the rating scale.

The Netflix Challenge

Why might a low-rank model work?

- Movies are not completely unrelated objects.
 - They have properties such as genre, style, cast, age and overall popularity.
- Users are not completely unrelated either.
 - They have different preferences for those properties, as well as different tendencies in how they use the rating scale.
- Perhaps both movies and users can be represented using a relatively small number of *latent factors*.

The Netflix Challenge

Why might a low-rank model work?

- Movies are not completely unrelated objects.
 - They have properties such as genre, style, cast, age and overall popularity.
- Users are not completely unrelated either.
 - They have different preferences for those properties, as well as different tendencies in how they use the rating scale.
- Perhaps both movies and users can be represented using a relatively small number of *latent factors*.
 - The factors are learned from the ratings; we do not tell the model that one of them should mean “likes action movies”.

Learning a low-rank model from observed ratings

SVD gives us the idea, but not quite the solution

- If the entire matrix $\mathbf{R}$ were known, a truncated SVD would give its best low-rank approximation in terms of the Frobenius norm.

Learning a low-rank model from observed ratings

SVD gives us the idea, but not quite the solution

- If the entire matrix $\mathbf{R}$ were known, a truncated SVD would give its best low-rank approximation in terms of the Frobenius norm.
- But most entries of $\mathbf{R}$ are unknown, and treating them as zeros would solve the wrong problem.

Learning a low-rank model from observed ratings

SVD gives us the idea, but not quite the solution

- If the entire matrix $\mathbf{R}$ were known, a truncated SVD would give its best low-rank approximation in terms of the Frobenius norm.
- But most entries of $\mathbf{R}$ are unknown, and treating them as zeros would solve the wrong problem.
- Instead, construct two smaller matrices:

$$\mathbf{P} \in \mathbb{R}^{n_{\text{users}} \times k}, \quad \mathbf{Q} \in \mathbb{R}^{n_{\text{movies}} \times k},$$

where k is the number of latent factors.

Learning a low-rank model from observed ratings

SVD gives us the idea, but not quite the solution

- If the entire matrix $\mathbf{R}$ were known, a truncated SVD would give its best low-rank approximation in terms of the Frobenius norm.
- But most entries of $\mathbf{R}$ are unknown, and treating them as zeros would solve the wrong problem.
- Instead, construct two smaller matrices:

$$\mathbf{P} \in \mathbb{R}^{n_{\text{users}} \times k}, \quad \mathbf{Q} \in \mathbb{R}^{n_{\text{movies}} \times k},$$

where k is the number of latent factors.

- The predicted rating is the dot product of a user's factors and a movie's factors:

$$\hat{r}_{ui} = \mathbf{p}_u^\top \mathbf{q}_i.$$

Learning a low-rank model from observed ratings

Only score the ratings that we actually observed

- Let Ω be the set of user–movies pairs for which Netflix supplied a rating.

Learning a low-rank model from observed ratings

Only score the ratings that we actually observed

- Let Ω be the set of user–movies pairs for which Netflix supplied a rating.
- We can learn $\mathbf{P}$ and $\mathbf{Q}$ by minimising:

$$\mathcal{L}(\mathbf{P}, \mathbf{Q}) = \sum_{(u,i) \in \Omega} \left(r_{ui} - \mathbf{p}_u^\top \mathbf{q}_i \right)^2.$$

Learning a low-rank model from observed ratings

Only score the ratings that we actually observed

- Let Ω be the set of user–movies pairs for which Netflix supplied a rating.
- We can learn $\mathbf{P}$ and $\mathbf{Q}$ by minimising:

$$\mathcal{L}(\mathbf{P}, \mathbf{Q}) = \sum_{(u,i) \in \Omega} \left(r_{ui} - \mathbf{p}_u^\top \mathbf{q}_i \right)^2.$$

- This is more generally called *matrix factorisation* rather than SVD:

Learning a low-rank model from observed ratings

Only score the ratings that we actually observed

- Let Ω be the set of user–movies pairs for which Netflix supplied a rating.
- We can learn $\mathbf{P}$ and $\mathbf{Q}$ by minimising:

$$\mathcal{L}(\mathbf{P}, \mathbf{Q}) = \sum_{(u,i) \in \Omega} \left(r_{ui} - \mathbf{p}_u^\top \mathbf{q}_i \right)^2.$$

- This is more generally called *matrix factorisation* rather than SVD:
 - there are no orthogonality constraints or explicit singular values;

Learning a low-rank model from observed ratings

Only score the ratings that we actually observed

- Let Ω be the set of user–movies pairs for which Netflix supplied a rating.
- We can learn $\mathbf{P}$ and $\mathbf{Q}$ by minimising:

$$\mathcal{L}(\mathbf{P}, \mathbf{Q}) = \sum_{(u,i) \in \Omega} \left(r_{ui} - \mathbf{p}_u^\top \mathbf{q}_i \right)^2.$$

- This is more generally called *matrix factorisation* rather than SVD:
 - there are no orthogonality constraints or explicit singular values;
 - and the loss is evaluated only over Ω .

Learning a low-rank model from observed ratings

Can we derive an update for one rating?

- Take one observed rating $(u, i) \in \Omega$ and define its residual:

$$e_{ui} = r_{ui} - \mathbf{p}_u^\top \mathbf{q}_i.$$

Learning a low-rank model from observed ratings

Can we derive an update for one rating?

- Take one observed rating $(u, i) \in \Omega$ and define its residual:

$$e_{ui} = r_{ui} - \mathbf{p}_u^\top \mathbf{q}_i.$$

- Its squared-error loss is:

$$\ell_{ui} = e_{ui}^2.$$

Learning a low-rank model from observed ratings

Can we derive an update for one rating?

- Take one observed rating $(u, i) \in \Omega$ and define its residual:

$$e_{ui} = r_{ui} - \mathbf{p}_u^\top \mathbf{q}_i.$$

- Its squared-error loss is:

$$\ell_{ui} = e_{ui}^2.$$

- What are $\frac{\partial \ell_{ui}}{\partial \mathbf{p}_u}$ and $\frac{\partial \ell_{ui}}{\partial \mathbf{q}_i}$?

Learning a low-rank model from observed ratings

Can we derive an update for one rating?

- Take one observed rating $(u, i) \in \Omega$ and define its residual:

$$e_{ui} = r_{ui} - \mathbf{p}_u^\top \mathbf{q}_i.$$

- Its squared-error loss is:

$$\ell_{ui} = e_{ui}^2.$$

- What are $\frac{\partial \ell_{ui}}{\partial \mathbf{p}_u}$ and $\frac{\partial \ell_{ui}}{\partial \mathbf{q}_i}$?
 - Start with one component, p_{uj} or q_{ij} , and use the chain rule.

Learning a low-rank model from observed ratings

The resulting gradients are rather simple

$$\frac{\partial e_{ui}}{\partial \mathbf{p}_u} = -\mathbf{q}_i,$$

$$\frac{\partial e_{ui}}{\partial \mathbf{q}_i} = -\mathbf{p}_u$$

Learning a low-rank model from observed ratings

The resulting gradients are rather simple

$$\frac{\partial e_{ui}}{\partial \mathbf{p}_u} = -\mathbf{q}_i,$$

$$\frac{\partial e_{ui}}{\partial \mathbf{q}_i} = -\mathbf{p}_u$$

$$\frac{\partial \ell_{ui}}{\partial \mathbf{p}_u} = -2e_{ui}\mathbf{q}_i,$$

$$\frac{\partial \ell_{ui}}{\partial \mathbf{q}_i} = -2e_{ui}\mathbf{p}_u.$$

Learning a low-rank model from observed ratings

The resulting gradients are rather simple

$$\frac{\partial e_{ui}}{\partial \mathbf{p}_u} = -\mathbf{q}_i,$$

$$\frac{\partial e_{ui}}{\partial \mathbf{q}_i} = -\mathbf{p}_u$$

$$\frac{\partial \ell_{ui}}{\partial \mathbf{p}_u} = -2e_{ui}\mathbf{q}_i,$$

$$\frac{\partial \ell_{ui}}{\partial \mathbf{q}_i} = -2e_{ui}\mathbf{p}_u.$$

Absorbing the factor of two into the learning rate, stochastic gradient descent gives:

Learning a low-rank model from observed ratings

The resulting gradients are rather simple

$$\frac{\partial e_{ui}}{\partial \mathbf{p}_u} = -\mathbf{q}_i,$$

$$\frac{\partial e_{ui}}{\partial \mathbf{q}_i} = -\mathbf{p}_u$$

$$\frac{\partial \ell_{ui}}{\partial \mathbf{p}_u} = -2e_{ui}\mathbf{q}_i,$$

$$\frac{\partial \ell_{ui}}{\partial \mathbf{q}_i} = -2e_{ui}\mathbf{p}_u.$$

Absorbing the factor of two into the learning rate, stochastic gradient descent gives:

$$\mathbf{p}_u \Leftarrow \mathbf{p}_u + \eta e_{ui} \mathbf{q}_i,$$

$$\mathbf{q}_i \Leftarrow \mathbf{q}_i + \eta e_{ui} \mathbf{p}_u.$$

Learning a low-rank model from observed ratings

One rating at a time

- Initialise the user and movies factors to small values.

Learning a low-rank model from observed ratings

One rating at a time

- Initialise the user and movies factors to small values.
- Repeatedly visit the ratings in Ω :

Learning a low-rank model from observed ratings

One rating at a time

- Initialise the user and movies factors to small values.
- Repeatedly visit the ratings in Ω :
 - 1 predict $\hat{r}_{ui} = \mathbf{p}_u^\top \mathbf{q}_i$;

Learning a low-rank model from observed ratings

One rating at a time

- Initialise the user and movies factors to small values.
- Repeatedly visit the ratings in Ω :
 - 1 predict $\hat{r}_{ui} = \mathbf{p}_u^\top \mathbf{q}_i$;
 - 2 compute the residual $e_{ui} = r_{ui} - \hat{r}_{ui}$;

Learning a low-rank model from observed ratings

One rating at a time

- Initialise the user and movies factors to small values.
- Repeatedly visit the ratings in Ω :
 - 1 predict $\hat{r}_{ui} = \mathbf{p}_u^\top \mathbf{q}_i$;
 - 2 compute the residual $e_{ui} = r_{ui} - \hat{r}_{ui}$;
 - 3 update only $\mathbf{p}_u$ and $\mathbf{q}_i$.

Learning a low-rank model from observed ratings

One rating at a time

- Initialise the user and movies factors to small values.
- Repeatedly visit the ratings in Ω :
 - 1 predict $\hat{r}_{ui} = \mathbf{p}_u^\top \mathbf{q}_i$;
 - 2 compute the residual $e_{ui} = r_{ui} - \hat{r}_{ui}$;
 - 3 update only $\mathbf{p}_u$ and $\mathbf{q}_i$.
- Each update touches only two vectors of length k .

Learning a low-rank model from observed ratings

One rating at a time

- Initialise the user and movies factors to small values.
- Repeatedly visit the ratings in Ω :
 - 1 predict $\hat{r}_{ui} = \mathbf{p}_u^\top \mathbf{q}_i$;
 - 2 compute the residual $e_{ui} = r_{ui} - \hat{r}_{ui}$;
 - 3 update only $\mathbf{p}_u$ and $\mathbf{q}_i$.
- Each update touches only two vectors of length k .
 - A complete pass costs $\mathcal{O}(k|\Omega|)$ rather than operating on all 8.5 billion possible entries.

Learning a low-rank model from observed ratings

One rating at a time

- Initialise the user and movies factors to small values.
- Repeatedly visit the ratings in Ω :
 - 1 predict $\hat{r}_{ui} = \mathbf{p}_u^\top \mathbf{q}_i$;
 - 2 compute the residual $e_{ui} = r_{ui} - \hat{r}_{ui}$;
 - 3 update only $\mathbf{p}_u$ and $\mathbf{q}_i$.
- Each update touches only two vectors of length k .
 - A complete pass costs $\mathcal{O}(k|\Omega|)$ rather than operating on all 8.5 billion possible entries.
- In an implementation, compute the two updates simultaneously or retain the old value of $\mathbf{p}_u$ for the update to $\mathbf{q}_i$.

Learning a low-rank model from observed ratings

Don't make the latent factors explain the obvious stuff

- First centre ratings around the global mean μ , so the factors do not need to learn the overall level of the rating scale.

Learning a low-rank model from observed ratings

Don't make the latent factors explain the obvious stuff

- First centre ratings around the global mean μ , so the factors do not need to learn the overall level of the rating scale.
- Some movies are broadly popular, and some users give consistently higher ratings than others.

Learning a low-rank model from observed ratings

Don't make the latent factors explain the obvious stuff

- First centre ratings around the global mean μ , so the factors do not need to learn the overall level of the rating scale.
- Some movies are broadly popular, and some users give consistently higher ratings than others.
- Add a global mean and user and movie biases:

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^\top \mathbf{q}_i.$$

Learning a low-rank model from observed ratings

Don't make the latent factors explain the obvious stuff

- First centre ratings around the global mean μ , so the factors do not need to learn the overall level of the rating scale.
- Some movies are broadly popular, and some users give consistently higher ratings than others.
- Add a global mean and user and movie biases:

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^\top \mathbf{q}_i.$$

- The latent factors can then focus on interactions between particular kinds of users and movies.

Learning a low-rank model from observed ratings

Regularisation is a separate modelling choice

- Mean-centering and bias terms capture systematic baseline effects; regularisation addresses a different problem.

Learning a low-rank model from observed ratings

Regularisation is a separate modelling choice

- Mean-centering and bias terms capture systematic baseline effects; regularisation addresses a different problem.
- For one observed rating, we can penalise large factor vectors:

$$e_{ui} = r_{ui} - \left(\mu + b_u + b_i + \mathbf{p}_u^\top \mathbf{q}_i \right),$$
$$\ell_{ui}^{\text{reg}} = e_{ui}^2 + \lambda \left(\|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2 \right).$$

Learning a low-rank model from observed ratings

Regularisation is a separate modelling choice

- Mean-centering and bias terms capture systematic baseline effects; regularisation addresses a different problem.
- For one observed rating, we can penalise large factor vectors:

$$e_{ui} = r_{ui} - \left(\mu + b_u + b_i + \mathbf{p}_u^\top \mathbf{q}_i \right),$$
$$\ell_{ui}^{\text{reg}} = e_{ui}^2 + \lambda \left(\|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2 \right).$$

- The corresponding factor updates gain a shrinkage term:

$$\mathbf{p}_u \Leftarrow \mathbf{p}_u + \eta(e_{ui}\mathbf{q}_i - \lambda\mathbf{p}_u), \quad \mathbf{q}_i \Leftarrow \mathbf{q}_i + \eta(e_{ui}\mathbf{p}_u - \lambda\mathbf{q}_i).$$

Learning a low-rank model from observed ratings

Regularisation is a separate modelling choice

- Mean-centering and bias terms capture systematic baseline effects; regularisation addresses a different problem.
- For one observed rating, we can penalise large factor vectors:

$$e_{ui} = r_{ui} - (\mu + b_u + b_i + \mathbf{p}_u^\top \mathbf{q}_i),$$
$$\ell_{ui}^{\text{reg}} = e_{ui}^2 + \lambda (\|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2).$$

- The corresponding factor updates gain a shrinkage term:

$$\mathbf{p}_u \Leftarrow \mathbf{p}_u + \eta(e_{ui}\mathbf{q}_i - \lambda\mathbf{p}_u), \quad \mathbf{q}_i \Leftarrow \mathbf{q}_i + \eta(e_{ui}\mathbf{p}_u - \lambda\mathbf{q}_i).$$

- Choose λ using validation data; regularisation is particularly useful for users and movies with few ratings.

Learning a low-rank model from observed ratings

Regularisation is a separate modelling choice

- Mean-centering and bias terms capture systematic baseline effects; regularisation addresses a different problem.
- For one observed rating, we can penalise large factor vectors:

$$e_{ui} = r_{ui} - \left(\mu + b_u + b_i + \mathbf{p}_u^\top \mathbf{q}_i \right),$$
$$\ell_{ui}^{\text{reg}} = e_{ui}^2 + \lambda \left(\|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2 \right).$$

- The corresponding factor updates gain a shrinkage term:

$$\mathbf{p}_u \Leftarrow \mathbf{p}_u + \eta(e_{ui}\mathbf{q}_i - \lambda\mathbf{p}_u), \quad \mathbf{q}_i \Leftarrow \mathbf{q}_i + \eta(e_{ui}\mathbf{p}_u - \lambda\mathbf{q}_i).$$

- Choose λ using validation data; regularisation is particularly useful for users and movies with few ratings.
 - The bias terms may also be regularised.

The Netflix Challenge

Where does the Simon Funk SVD fit?

- Simon Funk publicly described a remarkably simple gradient-based factorisation during the competition.

Sources: Funk (2006); Koren, The BellKor Solution to the Netflix Grand Prize (2009).

The Netflix Challenge

Where does the Simon Funk SVD fit?

- Simon Funk publicly described a remarkably simple gradient-based factorisation during the competition.
 - His implementation trained one latent feature at a time, cached residuals and added shrinkage to reduce overfitting.

Sources: Funk (2006); Koren, The BellKor Solution to the Netflix Grand Prize (2009).

The Netflix Challenge

Where does the Simon Funk SVD fit?

- Simon Funk publicly described a remarkably simple gradient-based factorisation during the competition.
 - His implementation trained one latent feature at a time, cached residuals and added shrinkage to reduce overfitting.
- It made the basic idea accessible and demonstrated that it could be both fast and memory efficient.

Sources: Funk (2006); Koren, The BellKor Solution to the Netflix Grand Prize (2009).

The Netflix Challenge

Where does the Simon Funk SVD fit?

- Simon Funk publicly described a remarkably simple gradient-based factorisation during the competition.
 - His implementation trained one latent feature at a time, cached residuals and added shrinkage to reduce overfitting.
- It made the basic idea accessible and demonstrated that it could be both fast and memory efficient.
- Matrix-factorisation models became important ingredients in leading solutions...

Sources: Funk (2006); Koren, The BellKor Solution to the Netflix Grand Prize (2009).

The Netflix Challenge

Where does the Simon Funk SVD fit?

- Simon Funk publicly described a remarkably simple gradient-based factorisation during the competition.
 - His implementation trained one latent feature at a time, cached residuals and added shrinkage to reduce overfitting.
- It made the basic idea accessible and demonstrated that it could be both fast and memory efficient.
- Matrix-factorisation models became important ingredients in leading solutions...
 - ...but the 2009 winning system was a large blend of many predictors, not Funk's algorithm alone.

Sources: Funk (2006); Koren, The BellKor Solution to the Netflix Grand Prize (2009).

The Netflix Challenge

The data were anonymous — weren't they?

- Netflix did not publish customer names or contact details.

The Netflix Challenge

The data were anonymous — weren't they?

- Netflix did not publish customer names or contact details.
- Each released record did, however, contain:
 - a pseudonymous customer identifier;
 - the movie that they rated;
 - the rating;
 - and the date on which they rated it.

The Netflix Challenge

The data were anonymous — weren't they?

- Netflix did not publish customer names or contact details.
- Each released record did, however, contain:
 - a pseudonymous customer identifier;
 - the movie that they rated;
 - the rating;
 - and the date on which they rated it.
- Is removing a person's name sufficient to make their data anonymous?

The Netflix Challenge

Sparse behaviour can form a fingerprint

- An individual is unlikely to have rated exactly the same unusual combination of movies, on roughly the same dates, as many other people.

The Netflix Challenge

Sparse behaviour can form a fingerprint

- An individual is unlikely to have rated exactly the same unusual combination of movies, on roughly the same dates, as many other people.
- Narayanan and Shmatikov showed how a little outside knowledge could be linked to the released Netflix records.
 - Public ratings from IMDb supplied examples of that outside knowledge.
 - The matching procedure tolerated approximate ratings, incorrect entries and errors in rating dates.

The Netflix Challenge

Sparse behaviour can form a fingerprint

- An individual is unlikely to have rated exactly the same unusual combination of movies, on roughly the same dates, as many other people.
- Narayanan and Shmatikov showed how a little outside knowledge could be linked to the released Netflix records.
 - Public ratings from IMDb supplied examples of that outside knowledge.
 - The matching procedure tolerated approximate ratings, incorrect entries and errors in rating dates.
- Once a record was linked to a known person, their other ratings could reveal preferences that they had never made public.

The Netflix Challenge

Pseudonymisation is not necessarily anonymisation

- Whether a dataset is anonymous depends on more than whether obvious identifiers have been removed.

The Netflix Challenge

Pseudonymisation is not necessarily anonymisation

- Whether a dataset is anonymous depends on more than whether obvious identifiers have been removed.
- We must also ask:
 - How distinctive is each person's pattern of behaviour?
 - What auxiliary information might an attacker possess now — or in the future?
 - Which attributes are actually required for the intended research?

The Netflix Challenge

Pseudonymisation is not necessarily anonymisation

- Whether a dataset is anonymous depends on more than whether obvious identifiers have been removed.
- We must also ask:
 - How distinctive is each person's pattern of behaviour?
 - What auxiliary information might an attacker possess now — or in the future?
 - Which attributes are actually required for the intended research?
- Could Netflix have reduced the risk by coarsening dates, suppressing rare ratings, aggregating records, or adding noise?
 - What would each of these do to the scientific utility of the data?

The Netflix Challenge

The two sides of sparsity

- The rating matrix was sparse, so we could train efficiently by updating only the two factor vectors involved in each observed rating.
- Individual behaviour was also sparse, so a small set of unusual ratings could make a person's record distinctive.
- The same property that made the learning problem computationally tractable helped create the privacy risk.